

**Leuphana University Lüneburg,
Summer Semester 2026**

30.09.2026

Module: Projekt Softwareentwicklung

Seminar: Information Extraction (S)

Instructors: Fatemeh Ghoochani,
Laurin Luttmann,

Portfolio Information Extraction

Marib Ahmad 4006933
Nigina Fazel 4005099
Wirtschaftsinformatik
marib.ahmad@stud.leuphana.de
nigina.fazel@stud.leuphana.de

Contents

1 Introduction	1
2 Background and Related Work	1
3. Method	2
3.1 Task definition and annotation layers	2
3.2 Transformer-Encoder and Fine-Tuning	2
3.3 Subword alignment	3
3.4 CRF-head and BIO-Decoding	3
3.5 Multi-task architecture	4
3.6 Evaluation	4
4. Data	5
4.1 SkillSpan dataset	5
4.2 Preprocessing and data quality	5
4.3 The German test set	6
5. Implementation	6
5.1 Technical implementation	6
5.2 Hyperparameters and training environment	7
5.3 Deviations from the original implementation	7
5.4 CRF implementation and debugging	8
5.5 Reproducibility	9
6. Experiments and Results	9
6.1 Experiment 1 – BERT vs. JobBERT	9
6.2 Experiment 2 – CRF vs. linear head	10
6.3 Experiment 3 – Separate models vs. multi-task	10
6.4 Experiment 4 - XLM-R zero-shot transfer	11
7. Error analysis	12
7.1 Error analysis on the English test set	12
7.2 Error analysis on the German test set	12
8. Discussion	13
8.1 Reproducing the SkillSpan results	13
8.2 Influence of the architecture components	13
8.3 Cross-lingual transfer	14
8.4 Limitations	14
8.5 Outlook	14
9. Conclusion	15
10. References	16

1 Introduction

Job postings show quite precisely which skills employers expect. That information sits in running text, though, and is worded in very different ways: *teamfähig und belastbar* ("a team player and resilient") and *Zusammenarbeit im multiprofessionellen Team* ("working in a multiprofessional team") can mean the same requirement without sharing a single keyword. A simple keyword search is therefore not enough, which makes automatic extraction a demanding information extraction (IE) task.

Zhang et al. (2022) distinguish between skills, meaning abilities and activities such as "managing client projects", and knowledge, meaning things a person knows, such as "Python" or "HGB" (German commercial accounting law). Because the two can overlap, they are annotated and recognized as two separate layers. Applications range from labor market analysis to matching applicants with vacancies and planning further training.

In this project we reimplemented SkillSpan (Zhang et al., 2022) and examined it in a set of ablation studies. We also built a German test set of 30 job postings, annotated twice independently and then reconciled. We pursue three questions: whether the central results of SkillSpan can be reproduced, what domain-specific pretraining, the CRF decoder and joint training of both layers each contribute, and how well a model trained only on English transfers to German postings (zero-shot).

2 Background and Related Work

Earlier approaches to skill extraction work with word lists, rules or existing ontologies such as ESCO. ESCO provides occupations, skills and competences in a structured form and can therefore serve as a basis for mapping skills (European Commission, n.d.). With freely worded postings, however, such approaches quickly reach their limits. The same skill can be described in many different ways, and not every formulation can be captured with fixed terms or rules. More recent approaches therefore use language models, which take the context of a formulation into account. The main basis for our work is SkillSpan by Zhang et al. (2022). The authors address skill extraction from English job postings and provide an annotated dataset for it. The full dataset described in the paper covers around 14,500 sentences with more than 12,500 annotated spans from three sources. Only two of these sources are publicly available, and those are the ones we work with (Section 4.1). Important for us is that the authors annotate two separate layers, SKILL and KNOWLEDGE (Section 3.1). We adopt this split for training and for our German test set. Besides the dataset, domain-adaptive pretraining plays an important role. BERT is a generally pretrained language model and was not developed specifically for job postings. Job postings, however, have their own typical terms and phrasings. In domain-adaptive pretraining, a pretrained model is therefore trained further on texts from a particular domain so that it adapts better to its language. Zhang et al. (2022) show that the adapted models perform considerably better than the

general ones. We take this up and compare BERT with JobBERT, which lets us check whether adapting to the job domain also pays off in our reimplementation.

A second focus is transfer between languages. For this we use XLM-RoBERTa, XLM-R for short. The model was pretrained on texts in 100 languages and is therefore well suited to transfer from one language to another (Conneau et al., 2020). We use it for our zero-shot experiment (Section 3.2): this lets us test not only the reimplementation of SkillSpan, but also how robust the approach is when the language changes.

3. Method

3.1 TASK DEFINITION AND ANNOTATION LAYERS

Like Zhang et al. (2022), we treat skill extraction as sequence labeling. Every word in a sentence gets a label, and the spans we are looking for can be read off from those labels. For this we use the BIO scheme (Ramshaw and Marcus, 1995). B marks the beginning of a span and I every further word inside it. O stands for everything outside a span. A sequence like O I is therefore invalid. A span simply cannot start with I.

SkillSpan distinguishes two layers. Skills describe what a person does and can do, for example *Analyse von Finanzdaten* ("analysis of financial data"). Knowledge describes knowledge and tools such as "Python" or "HGB". The two layers can overlap. In *Programmierung mit Python* ("programming with Python") the whole phrase is a skill, and "Python" is knowledge at the same time. With only one label set this could not be represented. Each word therefore gets two separate labels. This is not a rare edge case either. In our German test set, 23% of the knowledge spans lie inside a skill span.

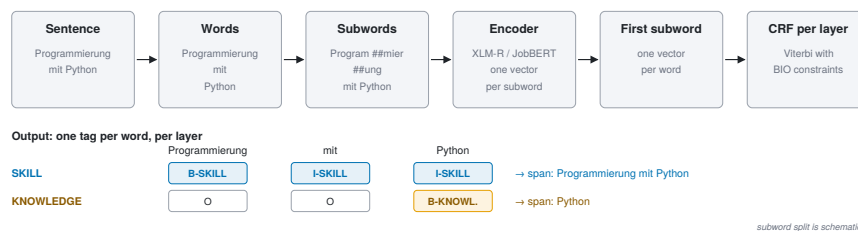


Figure 1. From sentence to label, Figure created with AI support.

3.2 TRANSFORMER-ENCODER AND FINE-TUNING

A pretrained transformer computes a representation for every word and takes the whole sentence into account while doing so. That matters. The same word can be a requirement in one context and not in another. With around 4,800 training sentences you could not train a model from scratch. We therefore adapt pretrained models by fine-tuning them.

We compare three models. BERT (Devlin et al., 2019) is a general language model. JobBERT (Zhang et al., 2022) is BERT with additional pretraining on job postings. Comparing the two shows

us what that pretraining is worth. XLM-R (Conneau et al., 2020) is a multilingual model. We train it only on English data and test it directly on German, since we have no German training data.

3.3 SUBWORD ALIGNMENT

Transformers do not work with whole words but with subwords. Rare or long words are split into smaller pieces. A long word like *Programmierung* can break apart into several subwords, while *mit* stays a single subword. The labels in the dataset, however, are assigned per word.

Like the original, we therefore use the first-subword strategy (Figure 2). Only the first subword of a word receives that word's label. All further subwords are masked and count neither for the loss nor for the prediction. That leaves exactly one label per word. This works well because, through self-attention, the first subword already carries information about the whole word and the sentence. For the CRF, however, the masking has an important consequence. We come back to that in Section 3.4.

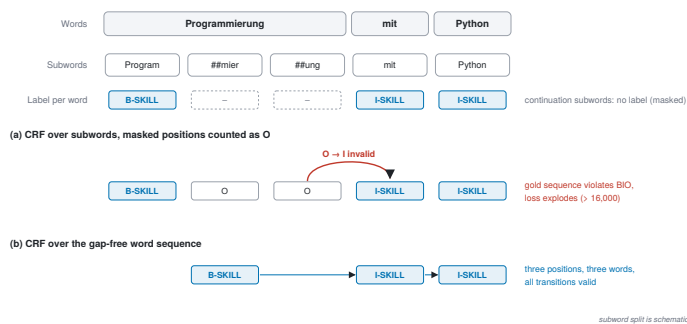


Figure 2. Subword alignment, and why the CRF has to run over words rather than subwords. Figure created with AI support.

3.4 CRF-HEAD AND BIO-DECODING

A simple linear head classifies each word on its own and does not look at its neighbors. Because of that it can produce invalid sequences such as O I. This happens, for example, when it misses the beginning of a span but still recognizes the end. A conditional random field (CRF, Lafferty et al., 2001), by contrast, scores the whole label sequence. It combines two kinds of scores for this. Emission scores say how well a label fits a word. A learned transition matrix says how plausible the change from one label to the next is. During training, the probability of the correct sequence is maximized. At prediction time, the Viterbi algorithm searches for the sequence with the highest overall score. On top of that, we rule out invalid transitions outright: a transition from O to I, and a sentence starting with I. The CRF therefore always outputs valid BIO sequences. What matters here is which positions the sequence runs over. The CRF has to run over the words, not over the subwords. Otherwise gaps appear at the masked positions, and if those gaps are counted as O, even the correct sequence becomes invalid (Section 5.4).

3.5 MULTI-TASK ARCHITECTURE

There are two ways to learn the two label layers. In the separate setup, each layer gets its own model with its own encoder and a CRF. In the joint setup, both layers share one encoder, with two heads on top, each with its own CRF (Figure 3). The losses of both heads go into training together, so the encoder is adapted for both tasks at the same time.

Skills and knowledge often refer to the same places in the text, as the example Programmierung mit Python shows. An encoder that serves both layers therefore gets more clues about where the technical requirements are. The joint setup is also more efficient: you train one encoder instead of two. Zhang et al. (2022), however, found that separate training worked better in their case. We check in a direct comparison whether the same holds for us.

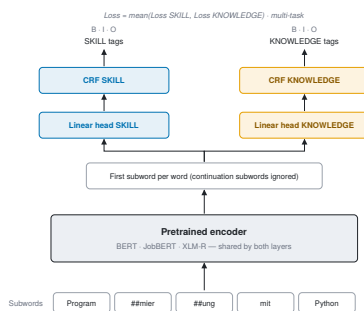


Figure 3. Multi-task setup: one shared encoder, two heads, each with its own CRF. Figure created with AI support.

3.6 EVALUATION

We evaluate the models at the level of whole spans with seqeval (Nakayama, 2018), separately for skills and knowledge. A predicted span only counts as correct if its beginning, its end and its type match a gold span exactly. From this we compute precision, recall and F1. Precision is the share of correct spans among all predictions. Recall is the share of gold spans that were found. F1 is the harmonic mean of the two. Token accuracy would be misleading here, because most words do not belong to any span at all. In our German test set, 94% of the words lie outside every knowledge span. A model that predicts nothing but O would therefore reach 94% token accuracy without having found a single span. The strict span metric is also quite harsh, though. A span with one word too many or too few counts as an error twice over: once as a false positive and once as a missed span. We look at how such boundary errors affect the results in Chapters 6 and 7.

To make the results reliable, we do three more things. First, some English test sentences appear word for word in the training data as well. Most of them are standard phrases. Good scores should not rest on simply recognizing such sentences again, so we additionally report F1 on unseen test sentences only. Second, we test every XLM-R model on both the English and the German test set. Model, configuration and seed stay the same, so the difference can be read directly as the transfer loss. Third, we train every configuration with three seeds and report the mean, the minimum and the maximum. We only count a difference between two configurations as a real effect if the value ranges do not overlap. That criterion is deliberately strict, so that we do not mistake random variation for a real effect.

4. Data

4.1 SKILLSPAN DATASET

We work with two datasets: the English SkillSpan dataset, on which we train and test all models, and our own German test set of 30 job postings. All data is in the `data/` folder of the repository.

SkillSpan. The English dataset comes from Zhang et al. and is published under an MIT license. It contains sentences from job postings with skill and knowledge spans, annotated as two separate BIO layers (Section 3.1). The postings come from two sources, labeled *tech* and *house* in the dataset. The paper describes a third source, but it was never released. Our absolute numbers can therefore not be compared directly with the paper. For our own comparisons this does not matter, because all models run on the same data.

The original files are kept unchanged in `data/raw/`. We prepare them with `prepare_data.py` and store the result in `data/processed/`.

	Training	Development	Test
Sentences	4,800	3,174	3,569
SKILL spans	2,221	1,070	1,091
KNOWLEDGE spans	2,969	1,093	1,174

German test set. The 30 German postings are stored as raw text in `data/raw_de/`. The test set comprises 623 sentences with 6,199 tokens and, in its final state, 324 skill spans and 244 knowledge spans. Because the model is only ever tested on this data, `data/processed_de/` contains the same English training and development data as `data/processed/`; only the test set is different. The guidelines, both annotations and the reconciliation are documented in `data/annotation/`.

4.2 PREPROCESSING AND DATA QUALITY

When we looked at the English data more closely, a few things stood out.

Some training entries are not sentences at all but whole paragraphs of up to 748 words, for example long lists of benefits. If you truncate those, spans are lost. We therefore split training entries with more than 100 words, and only at points where no span is cut in half. This turns 4,800 training sentences into 4,913. We leave the development and test sets unchanged so that the evaluation stays comparable with the paper. 3,283 of the 4,913 training sentences contain no span at all. They still matter, because the model also has to learn when to mark nothing.

812 test sentences appear word for word in the training set as well, mostly standard phrases such as "Job description". This is why we additionally compute F1 on the unseen test sentences only. 27 of these sentences are even annotated differently in training and in test. There the model learns a different solution from the one the test asks for, which puts a ceiling on the performance that can be reached. A few entries also consist of nothing but a control character, one in the training set and six

in the development set. After tokenization nothing of them is left. How we handle this is described in Section 5.4.

4.3 THE GERMAN TEST SET

So that the results would not hold for just one industry, we picked postings from as many different fields as possible: five from marketing and e-commerce, five from IT, data and engineering, eight from nursing, social work and education, two from human resources, and ten from other areas such as accounting, production or warehousing. For the annotation we wrote our own guidelines, based on those of SkillSpan. They define what counts as a skill and what as knowledge, where a span starts and ends, and what is not annotated at all, for example job titles, degrees, certificates and statements of willingness or availability. The postings were annotated twice, independently of each other. Annotation A was produced with AI support. Annotation B was produced by hand and afterwards summarized with the help of an AI. Before reconciliation, agreement at the token level was a Cohen's κ of 0.673 for skills and 0.664 for knowledge. The κ value therefore measures agreement between an AI-supported and a manual annotation, which should be kept in mind when interpreting it. We then went through the differences together and merged them into a final test set following fixed rules. As in English, the two layers differ considerably. A skill span is 5.9 tokens long on average, and only 15% consist of a single word. Knowledge spans are much shorter at 1.6 tokens on average, and 69% consist of just one word such as Python or HGB. This matters later for the evaluation, because with long spans it is far easier to get the boundaries wrong.

5. Implementation

5.1 TECHNICAL IMPLEMENTATION

The code is written in Python, using PyTorch and the transformers library from Hugging Face. For evaluation we use seqeval. The required libraries are listed in requirements.txt. The repository keeps the library code (src/), the executable scripts (scripts/), one JSON configuration per experiment (configs/), the data (data/) and everything generated (results/, fehleranalyse/) in separate folders. How to set it all up is described in README.md, details on the data in DATEN.md.

A run is started with a configuration and a seed: `python scripts/train.py --config configs/jobbert_skill.json --seed 3477689`

The script trains the model, picks the best epoch, tests it and appends the result as a row to results/all_runs.csv. With --data the data directory can be switched without changing the configuration. That is how we test the same model either on the English or on the German test set.

Sanity check. Before training for real, we used --smoke to check that the pipeline works. The model is trained on only 20 sentences, all of which contain at least one span, for 60 epochs with a batch size of 4. It should learn these sentences almost by heart, so the dev F1 has to approach 1.0. If it stays low, the problem is in the pipeline and not in the model. Smoke runs do not go into the evaluation.

Model selection. After every epoch we evaluate on the development data and store the state with the best dev F1. That state is what gets tested, not the model after the last epoch. For the multi-task model we take the mean of the dev F1 values of both layers.

5.2 HYPERPARAMETERS AND TRAINING ENVIRONMENT

We trained on Kaggle with an NVIDIA T4 GPU. Depending on the model, a run of twelve epochs takes about 26 to 30 minutes.

The learning rate, batch size, optimizer, dropout and the BIO constraints are taken from the original configurations of Zhang et al. The learning rate schedule, the sequence length and the number of epochs are our own choices (Section 5.3). The values are the same for all main experiments; the configurations differ only in encoder, decoder and label layer.

Parameter	Value
Learning rate	$1 \cdot 10^{-4}$
Batch size	32
Optimizer	AdamW, $\beta = (0.9, 0.99)$, weight decay 0.01
Learning rate schedule	linear warmup over 30% of the steps, then linear decay
Gradient clipping	norm 1.0
Dropout	0.2
Maximum sequence length	256 subwords
Epochs	12
Subword strategy	first subword per word
Decoder	CRF with BIO constraints

Table 5.1: Hyperparameters for all runs.

5.3 DEVIATIONS FROM THE ORIGINAL IMPLEMENTATION

Learning rate schedule. The original uses the MaChAmp framework with a slanted triangular schedule, different learning rates per layer and gradual unfreezing of the encoder layers. We use a simple linear schedule with warmup and train all layers from the start. That is available directly in transformers and has fewer knobs to turn. We cannot rule out an effect on the results, and we come back to this in the discussion.

Sequence length. The original truncates inputs after 128 subwords. Some training entries, however, are whole paragraphs of up to 748 words (Section 4.2), and truncating those would lose spans. We therefore split training sentences with more than 100 words already during preprocessing, and only at points where no span is cut in half. On top of that we raised the limit to 256 subwords.

Development and test data stay unchanged so that the evaluation remains comparable with the paper.

Our own CRF. The original uses the CRF from AllenNLP, but that library is no longer maintained. We therefore implemented a linear-chain CRF ourselves, with the same BIO constraints as

MaChAmp (Section 5.4).

Epochs. We train for 12 epochs instead of 20 because our GPU quota was limited. In most runs the best model was reached before epoch 12 anyway. A comparison with BERT and the same seed confirmed this: with 20 epochs the test F1 was 0.5116, with 12 epochs 0.5190. That very 20-epoch run (seed 3477689) is, however, part of the BERT mean in Experiment 1, while all other runs use 12 epochs. With the 12-epoch run instead, the gap between BERT and JobBERT would be 3.7 points rather than 3.9. This does not change the conclusion.

Seeds. The original averages over five seeds, we over only three because of compute time. This is why our criterion for a real effect is deliberately strict (Section 3.6).

Data. Only two of the three subsets of the SkillSpan dataset are publicly available (Section 4.1).

5.4 CRF IMPLEMENTATION AND DEBUGGING

Our CRF learns a matrix of scores for all transitions between two tags, plus one vector each for the start and the end of a sentence. During training, the negative log-likelihood of the correct sequence is minimized. The sum over all possible sequences is computed by the forward algorithm, the prediction is made by the Viterbi algorithm. In the multi-task model each layer has its own CRF, and the total loss is the mean of the two losses.

BIO constraints. Invalid transitions get a penalty of $-10,000$. Forbidden are an I after an O, an I after a span of a different class, and a sentence starting with I. Strictly speaking $-\infty$ would be the right value, but that produces NaN values in the forward algorithm. A very large negative value rules these sequences out just as effectively in practice.

The bug. Our first version did not train at all. After a few steps the loss climbed to over 16,000, and some batches aborted on the GPU with a device-side assert. The reason was the interplay of subword alignment and CRF. The CRF scores transitions between neighboring positions, and at first those positions were the subwords. Between two labeled positions, however, there are subwords without a label: word continuations, [CLS] and padding. If these are treated as O, then a skill like *Programmierung mit Python*, whose first word breaks into three subwords, turns into the sequence B O O I I (Figure 2). But the transition $O \rightarrow I$ is forbidden. The correct sequence was therefore practically impossible for the model, which is why the loss exploded.

The fix. The function `compact()` in `model.py` takes only the vector of the first subword for each word and pushes these vectors together into a sequence without gaps. The CRF now works on that sequence alone. The correct sequence is then B I I again, and all transitions are allowed. After decoding, the tags are written back to the right subword positions using a stored index.

Empty sentences. The crash on the GPU had a second cause. Some entries consist only of control characters such as `\u200d`, which the tokenizer removes entirely. The compacted sequence is then

empty, and accessing its last element lands at index -1 . We now replace such sentences with a sequence containing a single O token. Since they contain no spans, this does not change the results.

What we found interesting about this bug is that both parts were correct on their own. The masked subwords are no problem for a linear head, and the BIO constraints are right for a sequence of words. Only together do they produce a target sequence that the model cannot represent under its own rules. The high loss did not mean that the model was learning badly, but that we had given it a contradictory target.

5.5 REPRODUCIBILITY

Every run writes a row to `results/all_runs.csv` with the configuration, the seed, the best epoch, the runtime and all test scores. All numbers in Chapter 6 can be recomputed from it.

The name of a run comes from its configuration. Since we used `xlmr_skill` for both the English and the German test, we renamed those rows afterwards to `xlmr_skill_en`, `xlmr_skill_de` and `xlmr_skill_de_final`.

All sources of randomness are fixed through the seed (random, numpy, torch, CUDA). Even so, training on the GPU is not fully deterministic, because some CUDA operations sum in no fixed order. The small rounding differences grow over the course of training. On the English test set this is barely noticeable. On the small German test set, however, a repeat run with seed 3477689 gave 0.389 instead of 0.351, a difference of almost four points. This is why we always report the mean and the value range over three seeds, and never individual runs.

6. Experiments and Results

All numbers in this chapter are strict span F1 scores on the test set, averaged over three seeds (3477689, 4213916 and 8749520). We always give the value range. We only count a difference as an effect if the value ranges of the two configurations do not overlap (Section 3.6).

6.1 EXPERIMENT 1 – BERT VS. JOBBERT

The first question was whether it helps if the encoder has already been trained on job postings. JobBERT has the same architecture and size as BERT, so the only difference is the pretraining.

Model	F1	Range	Precision	Recall
BERT	0,510	0.501 – 0.518	0,566	0,464
JobBERT	0,549	0.529 – 0.561	0,575	0,526

Table 6.1: BERT vs. JobBERT, skill layer, mean over three seeds.

JobBERT is 3.9 points ahead of BERT, and the ranges do not overlap. The worst JobBERT run is therefore still better than the best BERT run. As described in Section 5.3, one run with 20 epochs is part of the BERT mean. Without it the gap would be 3.7 points, which does not change the result.

What is interesting is where the advantage comes from. Precision rises by barely a point, but recall by a good six points. JobBERT therefore mainly finds more skills, rather than drawing the boundaries of the ones it finds more precisely. That fits with the idea that, through its pretraining, JobBERT already knows the typical phrasings of job postings and therefore recognizes them as skills more often. This confirms the core finding of Zhang et al.

6.2 EXPERIMENT 2 – CRF VS. LINEAR HEAD

In the second experiment we checked whether the CRF is worth it, or whether a simple linear head that classifies each word on its own does just as well. Both variants use JobBERT as the encoder.

Decoder	F1	Range	Precision	Recall
CRF	0,549	0.529 – 0.561	0,575	0,526
linear	0,513	0.502 – 0.520	0,498	0,528

Table 6.2: CRF vs. linear head, JobBERT encoder, skill layer, mean over three seeds.

The CRF is 3.6 points better, and here too the ranges do not overlap. This time, however, the gain comes entirely from precision (+7.7 points), while recall stays practically the same. There is a good explanation for this: the linear head can produce invalid sequences, for example an I-SKILL without a preceding B-SKILL. Under the strict metric such broken partial spans count as errors. The CRF prevents them through the BIO constraints. This matters especially for long skill spans, because there a whole chain of transitions has to be right.

6.3 EXPERIMENT 3 – SEPARATE MODELS VS. MULTI-TASK

Here the question was whether it helps to learn the skill and the knowledge layer together in one model instead of training two separate ones.

Layer	separate	multi-task	difference
Skill	0.549 (0.529 – 0.561)	0.556 (0.546 – 0.573)	0.7
Knowledge	0.681 (0.678 – 0.685)	0.696 (0.688 – 0.703)	1.5

Table 6.3: Separate models vs. multi-task, JobBERT encoder, mean over three seeds.

For the knowledge layer the ranges do not overlap, so joint training brings a real advantage there. It comes from recall (0.739 – 0.752 instead of 0.706 – 0.733). The skill annotation probably helps the model to see where the technical content sits in the sentence in the first place. For the skill layer the ranges overlap clearly, so there we only see a tendency. What matters is that joint training does not hurt either layer. Our result therefore points in the opposite direction from Zhang et al., who found separate training to work better (Section 8.2). We also noticed that knowledge is consistently recognized about 13 points better than skills. This has to do with the length of the spans. In the English test set a skill span is 3.9 tokens long on average, a knowledge span only 1.8, and more than half of the knowledge spans consist of a single word such as Python or SAP. With a metric that

demands exact boundaries, such a short span is far easier to hit than a long description of an activity.

6.4 EXPERIMENT 4 - XLM-R ZERO-SHOT TRANSFER

In the last experiment we wanted to know how much performance is lost when a model is trained only on English data and then tested on German job postings without a single German training example. For this we used the multilingual model XLM-R and tested it on our own annotated test set with 30 postings and 324 skill spans.

Seed	English	German	Difference
3477689	0,542	0,351	-19.0
4213916	0,516	0,364	-15.3
8749520	0,548	0,337	-21.1
Mean	0,535	0,351	-18.5

Table 6.4: XLM-R on the English and the German test set, identical model, configuration and seed.

The switch to German costs around 18.5 points. Without German training data the model therefore reaches about two thirds of its English performance. On English, incidentally, XLM-R sits at 0.535, between BERT and JobBERT, and so loses less against the specialized model than we had expected. The more important finding came about more or less by accident. We measured the transfer twice: first on annotation A, that is, before the two annotations were reconciled, and then on the final test set. Texts, model and seeds were identical; the only thing that changed were the span boundaries, through our reconciliation rules.

Test set	Mean F1	Loss vs. English
Annotation A	0,239	-29.6
Final test set	0,351	-18.5

Table 6.5: The same transfer, measured on two annotation states.

The adjusted span boundaries alone raise the score by 11.2 points. That is clearly more than the variation between individual runs, which is around four points (Section 5.5). A good third of the loss we measured at first was therefore not due to the language at all, but to how the spans had been annotated. The model has learned the SkillSpan convention, that is, rather short spans without framing words. If the test set deviates from that, the strict metric counts it as an error, even though the model actually found the skill. The number we measure therefore depends heavily on how the annotation was done, and not only on how good the model is. We discuss what this means for our results in Chapter 8.

7. Error analysis

The numbers in Chapter 6 show how many errors the models make, but not which ones. We therefore took a closer look at the errors themselves. The basis is the saved predictions with seed 3477689: JobBERT and XLM-R on the English test set, and XLM-R on the German one. The script `fehleranalyse.py` assigns every gold span to an error category automatically:

	JobBERT EN	XLM-R EN	XLM-R DE
exact match	53.3%	50.2%	36.7%
missed	24.3%	27.7%	31.2%
boundary error	22.4%	22.1%	32.1%
false alarms (share of predictions)	16.7%	18.0%	5.2%

Table 7.1: Distribution of the error categories across the three prediction sets.

After that we went through up to 30 examples per category by hand and gave each one a short note and a finding. We only summarize the most important results here; the full tables and all examples are in the repository under `fehleranalyse/`.

7.1 ERROR ANALYSIS ON THE ENGLISH TEST SET

In English, many errors sit at the span boundaries, and very often the word "and" is involved. Sometimes the model pulls two skills together into one, for example "analytical and critical thinking skills" instead of "critical thinking". Sometimes it tears a gold span apart, as in "coach and develop your team". It also often cuts off the framing: "ready to face new challenges" becomes "face new challenges".

Many false alarms are not really errors. For JobBERT, in 20 out of 30 cases the prediction was a plausible skill that was simply missing from the gold annotation. That fits the contradictions in the data set described in Section 4.2.

7.2 ERROR ANALYSIS ON THE GERMAN TEST SET

In German it is mainly the boundary errors that go up, from around 22 to 32%. Missed spans rise by only about four points, and false alarms are much rarer. The model often breaks spans off too early, above all before prepositions, so that "Wertberichtigungen nach HGB" comes out as just "Wertberichtigungen". What gets missed most are nominalizations such as "Weiterentwicklung und Optimierung des Investitionscontrollings".

Part of the errors goes back to our own annotation, though. Soft skill lists like "Zuverlässigkeit, Sorgfalt und Teamfähigkeit" we annotated as a single span, while the model splits them into individual skills, the way SkillSpan does. On top of that there are commas: the English test set has almost none, the German one has many, and the model often breaks off at them.

To check this with a number, we additionally computed a relaxed F1 with `locker_f1.py`, where a span already counts if it overlaps a gold span. Strictly evaluated, XLM-R reaches 0.541 on English and 0.389 on German. Under the relaxed metric the two are 0.769 and 0.798, so the gap disappears completely. Relaxed precision in German is 0.948, which means almost every prediction does hit a real skill; relaxed recall is 0.688 against 0.723 in English. The transfer loss is therefore almost entirely a boundary problem, not a recognition problem.

8. Discussion

8.1 REPRODUCING THE SKILLSPAN RESULTS

The first research question was whether we could reproduce central results from SkillSpan with our own implementation. What mattered to us was less hitting the exact same F1 scores than seeing whether the basic differences between the model variants show up for us as well. The clearest case is BERT against JobBERT: JobBERT is about 3.9 points better, and the ranges of the individual runs do not overlap. Domain-specific pretraining therefore pays off in our setup too. We would still not call this an exact reproduction, since we did not have all of the original data and our implementation and training scope differ in several places. We see our results as confirming the basic pattern, not the exact numbers.

8.2 INFLUENCE OF THE ARCHITECTURE COMPONENTS

The second question was what the individual parts of the architecture contribute. The CRF makes a clear difference of about 3.6 points, and the interesting part is where that comes from, precision rises while recall stays almost the same. The CRF does not make the model find more skills, it makes the spans it predicts correct more often. That fits how it works. Even with the linear head the words are not seen without context, because the JobBERT representations already carry context. But only the CRF also looks at how consecutive BIO labels fit together, and the labels have to add up to a valid span in the end. The higher precision suggests that it rules out impossible label sequences.

Multi-task learning is weaker. For SKILL the gain is under one point and the seeds overlap, so we only read it as a tendency. For KNOWLEDGE the ranges do not overlap, so by our criterion it counts as a real effect. That points in the opposite direction from Zhang et al. (2022), where separate training worked better; the differences in implementation and in the available data are the likely reason.

KNOWLEDGE also reaches higher F1 than SKILL overall. We think this is about span length: SKILL spans are 3.9 tokens long on average, KNOWLEDGE spans only 1.8, and more than half of the KNOWLEDGE spans are a single word. Under strict evaluation a long span simply offers more places to put a boundary wrong.

8.3 CROSS-LINGUAL TRANSFER

The third question was the transfer from English to German postings. XLM-R loses about 18.5 points, a relative drop of around 34%. Part of what was learned on English does carry over, but the performance is clearly worse: the model misses more spans and at the same time makes more predictions that are wrong or not exactly right. We cannot put that loss down to the language alone. Two observations point to the annotation. First, the same model with the same seeds scored 0.239 on annotation A and 0.351 on the final test set - 11.2 points purely from the adjusted span boundaries. Second, the gap between English and German almost disappears once an overlapping span already counts as a hit (Section 7.2). The model usually finds the right places and draws the boundaries differently than our annotation does. Language and annotation still cannot be separated completely, because the German postings are different texts annotated under our own guidelines.

For the research question this means that zero-shot transfer does work, but at a real cost. XLM-R's multilingual pretraining is not enough to reach the English level in German.

8.4 LIMITATIONS

Our results should be read within a few clear boundaries.

Only two of the three SkillSpan subsets are publicly available, and we trained for 12 epochs instead of 20 because of our GPU quota, so our absolute F1 values cannot be compared with the published ones directly. Our comparisons are between our own configurations, which all run on the same data. In the data, 812 test sentences appear word for word in the training set, and 27 of them are annotated differently there. This is why we report F1 on unseen test sentences alongside the standard score; the two stay close, so the overlap raises the absolute values somewhat but does not drive the differences we build on. We average over three seeds rather than five, and training on the GPU is not fully deterministic: a repeat run with the same seed differed by almost four points on the small German test set (Section 5.5). Differences of several points are safe on this basis, but a gain well under one point, such as the SKILL result from multi-task learning, stays a tendency.

Our German test set is 30 postings with 324 skill spans, so the zero-shot results describe this test set rather than German job advertisements in general. And because language and annotation guidelines change at the same time, the two effects can be narrowed down (Section 8.3) but not fully separated.

8.5 OUTLOOK

The obvious next steps are a larger German test set, and a comparison of pure zero-shot against a model that is additionally fine-tuned on a small amount of German data, to see whether a few examples already make a large difference. Evaluating relaxed metrics systematically for all models, rather than only diagnostically as in Section 7.2, would show how many errors are really just

boundary errors. More seeds and a BERT baseline for KNOWLEDGE would make the component comparison more complete.

9. Conclusion

The aim of our work was to reimplement the SkillSpan approach for the automatic extraction of skills and knowledge from job advertisements and to investigate it in more detail. We focused on three main questions: whether the central findings of SkillSpan could be reproduced, how individual architectural components influence the results, and how well a model trained only on English data transfers to German job advertisements. To investigate these questions, we compared different model variants and additionally created our own German test set of 30 job advertisements, which the two of us annotated independently and then reconciled into a final test set. Our results show that important trends from SkillSpan can also be observed in our reimplementation. The difference between BERT and JobBERT was particularly clear. JobBERT achieved an average F1 score of 0.549, compared with 0.510 for BERT, so domain-specific pretraining on job advertisements is beneficial for skill extraction in our experiments as well. The CRF also improved the results: compared with the linear head, the F1 score increased from 0.513 to 0.549, and this improvement came almost entirely from higher precision, while recall remained unchanged. Multi-task learning was the one component where we differ from the paper. For the knowledge layer, training both layers together was measurably better, 0.696 against 0.681, while Zhang et al. found separate training to work better; for the skill layer the difference was too small to count as an effect under our criterion.

The zero-shot transfer to German job advertisements was more challenging. XLM-R achieved an average F1 score of 0.535 on the English data and 0.351 on our final German test set, a decrease of about 18.5 F1 points. However, our experiments also showed that this difference cannot be explained by language alone. After reconciling the annotations and adjusting the span boundaries, the German F1 score rose from 0.239 to 0.351, and in our error analysis the gap between English and German almost disappeared once overlapping spans were counted as matches. XLM-R therefore often identifies the relevant parts of the text, but does not always predict the exact boundaries our annotation requires. The strict F1 score remains our main metric, but it does not show the complete picture of how well the model finds relevant skills.

Overall, our project shows that domain-specific pretraining and the choice of decoder have a measurable influence on skill extraction. At the same time, our zero-shot experiment shows that transferring a model to another language depends not only on the multilingual model itself, but also on the data and on the way spans are annotated. In future work, a larger German test set and additional German training data would help to examine the cross-lingual transfer more closely and to separate language-related effects from differences caused by the dataset and the annotation.

10. References

Zhang, M., Jensen, K. N., Sonniks, S. D., & Plank, B. (2022). SkillSpan: Hard and soft skill extraction from English job postings. *Proceedings of NAACL-HLT 2022*, 4962–4984.